

CSIS 2300 Introduction to Algorithms

Name: _____

Spring 2026 – Exam 3

RIN: _____

RCS username: _____

Instructions: You have 110 minutes to complete the exam. **Please present your ID when turning in your exam paper.**

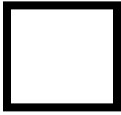
Be sure to show complete answers and all appropriate work **on the exam paper**; no scratch paper will be collected. You may use the back of any page (including this cover sheet) if you need more space. When continuing onto another page, be sure to **label the question number** on your work.

You are permitted your own paper notes, plus printouts of any course materials provided by the instructor (e.g. slides, assignments). During the exam, use of any electronic devices or other non-approved resources is prohibited, as is communicating with anyone other than the instructional staff. Violations of exam policies are grounds for failure of the course.

This examination consists of problems numbered 1-4; be sure to read all questions before beginning your exam.

Leave the boxes blank! They are for the graders!

The problems begin on the next page.



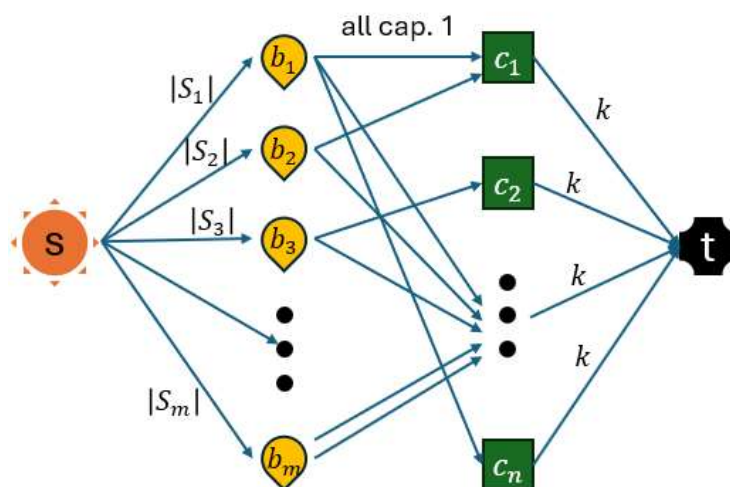
(1) [25 points] A research team wants to measure n different atmospheric conditions $C = \{c_1, \dots, c_n\}$ at a certain location. They have a collection of m weather balloons, but all of them are different. Each balloon has a specific set $S_i \subseteq C$ of conditions it can measure. (For example, balloon 1 might measure ozone, methane, and temperature; balloon 2 might measure only ozone and sulfur dioxide; etc.)

For reliability, the scientists want k measurements of each condition; further, they must come from different balloons. (i.e. Balloons can't measure the same thing twice.)

Describe how to set up a flow network such that finding the maximum flow will tell us if it is possible to meet the research team's requirements. Be sure to explain the meaning of each node and edge in your network, as well as why maximum flow will answer the question.

The intended graph has two layers: one with m nodes (1 per balloon) and one with n nodes (1 per conditions). We draw a capacity-1 edge from balloon i to condition j iff $c_j \in S_i$; this guarantees that balloons will only measure what they are capable of, and only once each. We add links from the source to each balloon with capacity $|S_i|$, so that each balloon can potentially perform all of its measurements. (You could just use n on the capacities here, since the outflow is already limited.) Finally, connect each condition node to the sink with capacity k .

Each unit of flow represents one successful measurement – the center edges ensure that this matches up one balloon to one condition. If the max flow is equal to kn , then we have achieved the goal – we have not measured any condition more than k times, so the flow (and thus measurements) must be spread equally among all of the conditions. Conversely, if the max flow is $< kn$, the goal must be impossible, since there is a cut (bottleneck) with lower capacity.



Grading notes: 15 points for the graph + algorithm; 10 points for explanation.

Graph: -2/layer for wrong capacities, -2 for not explaining connections, -2 for missing source/sink, -5 for correct picture with no explanation, -1 or -2 for other minor errors

Explanation: -5 for a “one way” explanation ($\Rightarrow$ instead of $\Leftrightarrow$), -2 for missing “return value”, -1 or -2 for other minor errors

(2) A television production company has studios in Manhattan and Hollywood. They have a show that can be shot in either location. The costs of shooting in each location change each week depending on a variety of factors. Let m_i and h_i represent the costs of shooting in Manhattan and Hollywood, respectively, in week i . There is additionally a fixed cost of c incurred each time shooting changes locations. Shooting can start in week 1 in either location without penalty.

The goal is to plan the locations for n weeks of shooting this show in such a way to minimize the total production costs.

Give a dynamic programming algorithm to solve this problem.

Big hint: This problem is effectively a 1-D problem, but you need two 1-D arrays. (You can also set it up as a single $2 \times n$ array.)

(a) [8 points] Describe the recursive substructure of this problem – how does the answer to the larger problem depend upon the answer to the smaller problems? What is the small choice that is made at each step of the process?

(b) [6 points] Write the relevant recurrence relation(s). (You might write one for each city, or just one that takes the city as an input.)

(c) [6 points] Write the pseudocode for your algorithm.

(d) [6 points] Is this algorithm a good candidate for memoization? Justify your answer.

(a) Clearly the key decision is where to shoot. In any span of days from 1 to i , examine the choice made on the last day. If we shoot in Manhattan on day i , the optimal cost will depend upon which is lower: the optimal cost ending in Manhattan on day $i - 1$; or, the switching cost plus the optimal cost ending in Hollywood on day $i - 1$. If we shoot in Hollywood on day i , it's just the reverse.

$$(b) \text{OPT}_M(i) = m_i + \min(\text{OPT}_M(i-1), c + \text{OPT}_H(i-1))$$

$$\text{OPT}_H(i) = h_i + \min(\text{OPT}_H(i-1), c + \text{OPT}_M(i-1))$$

$$\text{OPT}_M(0) = \text{OPT}_H(0) = 0$$

```
def location(n,c,m,h): # m & h are arrays with indices 1 through n
    Declare arrays OPTm and OPTh as length n+1
    OPTm[0] = OPTh[0] = 0
    for i = 1 to n:
        OPTm[i] = m[i] + min(OPTm[i-1], c + OPTh[i-1])
        OPTh[i] = h[i] + min(OPTh[i-1], c + OPTm[i-1])
    return min(OPTm[n],OPTh[n])
```

(d) No – the final values depend upon every previous value, so memoization doesn't buy us anything.

Grading notes:

(a) 4 points for relating larger answer to smaller subproblem, 4 points for choice made at each step of the process

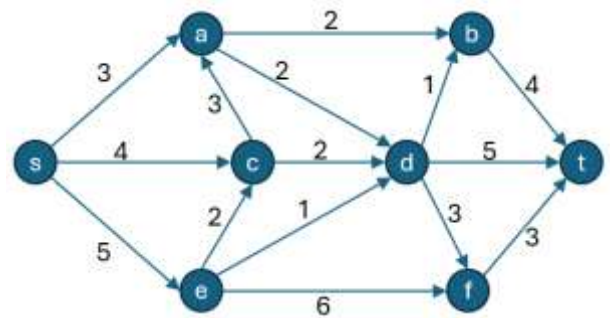
(b) -2 for missing base cases; -2 for trying to perform a summation in the recurrence

(c) 0 if the code is recursive and not DP (the only recursion acceptable would be with memorization); -2 for using $OPT[i - 1] = m_{i-1}$ as the location check; -4 if the loop(s) happen *inside* the check for location (i.e. we only look at location once)

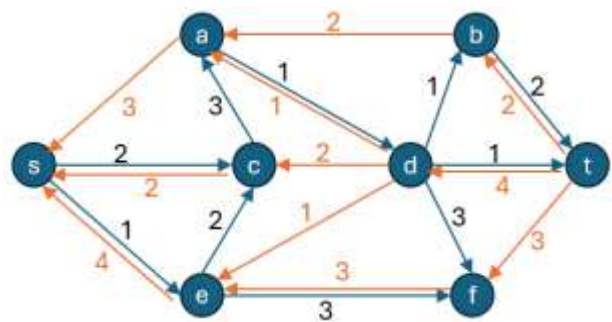
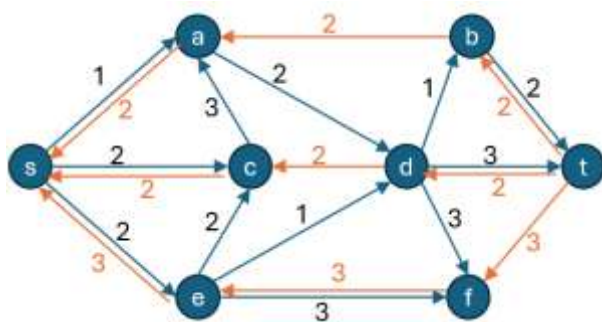
(d) 2 points for “no”; 4 points for “because we need to calculate every subproblem”

A coherent reason to use DP (as opposed to a reason to use memoization) is worth 3/6

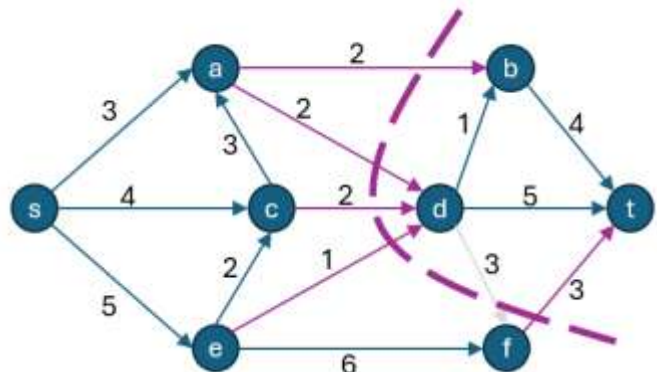
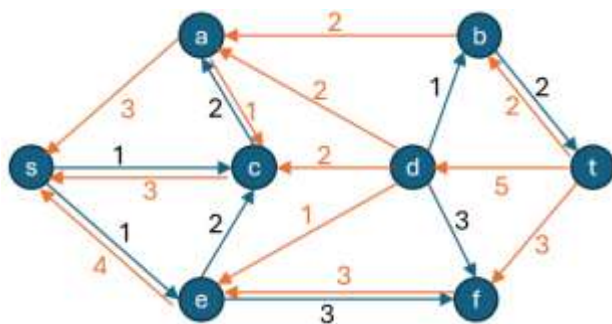
(3) [25 points] Execute the Edmonds-Karp variant of the Ford-Fulkerson procedure to find both the maximum flow and minimum cut on the flow network at right. To get full credit, you must state each augmenting path and draw the new residual graph after each iteration. **Don't forget** to also state the minimum cut at the end.



The first three paths are likely $s-a-b-t$, $s-c-d-t$, and $s-e-f-t$. After those we get the residual graph on the left. There are two other length-3 paths: $s-a-d-t$ and $s-e-d-t$. (Altogether, these must be the first five paths chosen, though not necessarily in this order.) In this order, after these five we get the residual on the right.



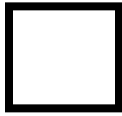
The last path chosen here will be $s-c-a-d-t$, resulting in the residual below left. The nodes a, c, e, f are reachable from s , and so we get the cut shown below right, with a capacity of 10.



Grading notes: 13 points for correct five augmenting paths; 12 points for everything else

Paths: -2 per incorrect/missing intermediate path (to max of -5); -5 if the final residual graph is missing entirely; -1 per incorrect edge in final residual, to max of -3

Everything else: -4 for missing mincut entirely; -2 for wrong max flow/min cut; -1 or -2 for other minor errors



(4) [25 points] Consider a service that processes high-throughput data streams. When data arrives at the service, it must be processed that hour; any unprocessed data is discarded as too old. (Besides, there's more coming in!) It is known how much data will arrive each hour for the next n hours: call these values $d_1, d_2, \dots, d_n$.

The machine doing the processing starts out fast, but over time it slows down in a predictable fashion until it is rebooted (memory leak?). Let s_i be the maximum amount of data the server can process in the i th hour after a reboot, with $s_1 \geq s_2 \geq \dots \geq s_n > 0$. Note that we can only process data that actually arrives: If the machine can do 10 units of data in the current hour, but only 8 of them arrive, then we're only going to process 8 units. (To repeat: No backlogs – old data is useless.)

Reboots take a full hour; during that hour no data gets processed. After a reboot, the machine can handle s_1 units of data again, and then it degrades as before. We assume that the machine was rebooted in hour 0 (i.e. just before our problem begins).

The task is to schedule reboots so as to maximize the amount of data processed in hours 1 through n .

As an example, suppose $n = 4$, $d = [10, 1, 7, 5]$ and $s = [8, 4, 2, 1]$. Then the optimal plan is to reboot only in hour 2 – this will allow us to process $8 + 0 + 7 + 4$ units of data, for a total of 19.

Develop a dynamic programming algorithm for this task. Your response must:

- (a) clearly state how larger subproblems depend on smaller ones
- (b) state the relevant recurrence relation
- (c) describe the DP table and state the order in which it will be filled
- (d) explain how to recover an actual reboot schedule from the table data

Obviously, your answer will continue on the next page.

Note: There are multiple valid ways to set this up; some with a 1-D table and some with a 2-D table; you may use either. I find the 2-D approaches more intuitive; the second variable tracks hours since the last reboot and has a triangular table. This approach runs in $O(n^2)$ time.

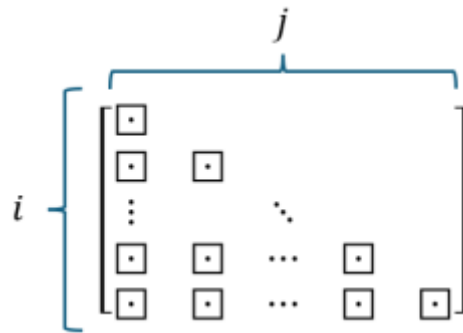
(a) There are multiple ways to do this. I expect most folks to pursue the line I hinted at.

If we do not reboot in hour i , there are no decisions to be made – we just process the data we can. The amount depends on when our last reboot was. We can record the optimal value for every possible time j since the last reboot.

If we do reboot in hour i , then our amount of data processed will not change from hour $i - 1$. In that case, our optimal value will simply be the best result from the previous hour, selecting from all possible times for the previous reboot.

(b) $OPT(i, j) = \min(s_j, d_i) + OPT(i - 1, j - 1); OPT(i, 0) = \max_{1 \leq j \leq i-1} OPT(i - 1, j); OPT(0, 0) = 0$

(c) The recurrence above sets up a 2-D table, but we only fill in the locations where $i \geq j$, so it's triangular. The key factor is that we must use a particular order: We need to fill all of the boxes for a particular value of i before moving on to $i + 1$. So, for example, if rows are i and columns are j , we get a lower-triangular array where we must complete each row before moving to the next row.



Each row of the array represents hour i ; the columns are how many hours j it has been since the last reboot.

To find the overall optimum answer, we take the maximum of the final row.

(d) Once we have this table completed, recovering the reboot schedule is very straightforward. Say the optimal answer occurs at $OPT(n, k)$. Then our last reboot took place in hour $n - k$. We can then look at hour $n - k - 1$, find its highest value, and repeat the process: Whichever column the best results occupies tells us how many hours back the previous reboot was.

Grading notes: The recurrence relation is the most important item, thus the breakdown was 5 points for (a), 14 points for (b), 3 points for (c), and 3 points for (d).

For (a): 3/5 for correctly describing the subproblems but having the dependence wrong

For (b): The critical insight is recognizing the maximization needed to find $OPT(i, 0)$; this section of the recurrence was worth 7 points. The first portion of the recurrence $OPT(i, j)$ was worth 4 points and the $OPT(0, 0)$ base case was worth 3 points.

For (c) and (d): Minor errors were -1 or -2